

Министерство образования Республики Беларусь
Учреждение Образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ
Кафедра программного обеспечения информационных технологий

Лабораторная работа № 6
« Защищенный и реальный режим процессора»

Проверил:Марзалюк А.В.
Выполнил: Новицкий Е.Ю

Минск 2026

Задание:

Написать программу, которая выполняет следующие действия:

- Переход из реального режима в защищенный.
- Перехватывает аппаратное прерывание от клавиатуры и заданное аппаратное прерывание, в обработчике которого выполняет определенные действия. Прерывание от клавиатуры реализуют все, номер аппаратного прерывания задается по вариантам (два варианта).
-
- По наступлению определенного события выполняет обратный переход из защищенного режима в реальный и завершает свою работу.

Для прерываний от таймера и часов реального времени обработчик прерывания должен отслеживать количество вызовов прерывания и отсчитывать секунды, выводя их на экран. Количество секунд после которых выполняется обратный переход в реальный режим и выход из программы (то самое определенное событие) считывается с клавиатуры перед переходом в защищенный режим..

Для прерывания от клавиатуры необходимо считывать скан-коды клавиш и выводить их на экран. По нажатию определенной клавиши (любой на выбор студента) осуществляется обратный переход в реальный режим и выход из программы.

При выполнении данной лабораторной работы должны быть соблюдены следующие условия:

- После завершения работы программы компьютер должен продолжать корректно функционировать. Зависания, перезагрузки и другие аналогичные «события» недопустимы;
- Переход в защищенный режим процессора должен быть выполнен по алгоритму, используемому в процессорах начиная с 386. Переход в защищенный режим с использованием алгоритма для 286 процессора недопустим.

Код программы\NASM:

```
.386P
.MODEL    LARGE

; CODE SEGMENT
CODE_RM segment para use16           ;16-digit segment starts
at address multiple 16               ;начало сегмента кода
CODE_RM_BEGIN    = $                 ;начало сегмента кода
реального режима                     ;point which segment load
    assume cs:CODE_RM,DS:DATA,ES:DATA
to segment regs
START:
    mov ax,DATA
    mov ds,ax
    mov es,ax

    lea dx,MSG_ENTER                  ;вывод сообщения
    mov ah,9h
    int 21h

    call INPUT
    mov ds:[TIME], al                ; кладем время в time

    call FILL_CR_0_BUFFER_RM          ;сформировать буфер cr0
    lea dx, BUFFER_CR_0_RM           ; вывести значение cr0 на
экран
    mov ah, 9h
    int 21h

    lea dx,MSG_HELLO                  ; попросить ввести любой
СИМВОЛ
    mov ah,9h
    int 21h

    mov ah,7h                         ;считать символ без эха
    int 21h

PREPARE_RTC:                          ;Подготовка часов RTC
    mov al,0Bh
    out 70h,al                       ;Выбрать регистр состояния
0Bh, в порт 70 кладем нужный регистр, в 71 - его значение
    in  al,71h
    or  al,00000100b                 ;Установить бит DM в 1 -
формат представления время в двоичном виде, сохранив предыдущую
конфигурацию
    out 71h,al                       ;Двоичный режим - для
простоты подсчета, чтоб не в BSD

; ЭТАП 2. Открыть линию A20.
ENABLE_A20:                           ;Открыть линию A20 (to get
more address memory)
    in  al,92h
    or  al,2                         ;Установить бит 1 в 1
    out 92h,al
```

```

; ЭТАП 3. Сохранить маски PIC.
SAVE_MASK:                                ;Сохранить маски
прерываний, тк мы будем менять обработчики прерываний
    in     al,21h
    mov     INT_MASK_M,al                  ;master
    in     al,0A1h
    mov     INT_MASK_S,al                  ;slave

; ЭТАП 1. Запретить все прерывания (cli, NMI).
DISABLE_INTERRUPTS:                        ;Запрет маскируемых и
немаскируемых прерываний
    cli                                       ;Запрет маскируемых
прерываний
    in     al,70h
    or     al,10000000b                    ;Установить 7 бит в 1
для запрета немаскируемых прерываний
    out    70h,al
    nop                                       ;skip tackt

; ЭТАП 4. Заполнить GDT линейными адресами (базами) сегментов.
LOAD_GDT:                                  ;Заполнить глобальную
таблицу дескрипторов
    mov     ax,DATA                        ;Формирование базового
адреса в сегменте данных
    mov     dl,ah
    xor     dh,dh
    shl     ax,4                            ;формирование 20-
разрядного адреса физической памяти(линейного адреса)
    shr     dx,4
    mov     si,ax                           ;si - младшая часть
    mov     di,dx                           ;di - старшая часть

WRITE_GDT:                                 ;Заполнить дескриптор GDT
    lea     bx,GDT_GDT                    ;смещение дескриптора GDT
    mov     ax,si
    mov     dx,di
    add     ax,offset GDT
    adc     dx,0                            ;учесть перенос
    mov     [bx][S_DESC.BASE_L],ax
    mov     [bx][S_DESC.BASE_M],dl
    mov     [bx][S_DESC.BASE_H],dh

WRITE_CODE_RM:                             ;Заполнить дескриптор
сегмента кода реального режима
    lea     bx,GDT_CODE_RM
    mov     ax,cs
    xor     dh,dh
    mov     dl,ah
    shl     ax,4
    shr     dx,4
    mov     [bx][S_DESC.BASE_L],ax
    mov     [bx][S_DESC.BASE_M],dl
    mov     [bx][S_DESC.BASE_H],dh

WRITE_DATA:                               ;Записать дескриптор
сегмента данных
    lea     bx,GDT_DATA
    mov     ax,si
    mov     dx,di
    mov     [bx][S_DESC.BASE_L],ax
    mov     [bx][S_DESC.BASE_M],dl
    mov     [bx][S_DESC.BASE_H],dh

```

```

WRITE_STACK:                                ;Записать дескриптор
сегмента стека
    lea bx, GDT_STACK
    mov ax, ss
    xor dh, dh
    mov dl, ah
    shl ax, 4
    shr dx, 4
    mov [bx][S_DESC.BASE_L], ax
    mov [bx][S_DESC.BASE_M], dl
    mov [bx][S_DESC.BASE_H], dh
WRITE_CODE_PM:                              ;Записать дескриптор кода
защищенного режима
    lea bx, GDT_CODE_PM
    mov ax, CODE_PM
    xor dh, dh
    mov dl, ah
    shl ax, 4
    shr dx, 4
    mov [bx][S_DESC.BASE_L], ax
    mov [bx][S_DESC.BASE_M], dl
    mov [bx][S_DESC.BASE_H], dh
WRITE_IDT:                                  ;Записать дескриптор IDT
    lea bx, GDT_IDT
    mov ax, si
    mov dx, di
    add ax, OFFSET IDT
    adc dx, 0
    mov [bx][S_DESC.BASE_L], ax
    mov [bx][S_DESC.BASE_M], dl
    mov [bx][S_DESC.BASE_H], dh
    mov IDTR.IDT_L, ax
    mov IDTR.IDT_H, dx
; ЭТАП 5. Заполнить IDT адресами обработчиков.
FILL_IDT:                                   ;Заполнить таблицу
дескрипторов векторов прерываний
    irpc N, 0123456789ABCDEF               ;Заполнить векторы 00-0F
исключениями
        lea eax, EXC_0&N
        mov IDT_0&N.OFFS_L, ax
        shr eax, 16
        mov IDT_0&N.OFFS_H, ax
    endm
    irpc N, 0123456789ABCDEF               ;Заполнить векторы 10-1F
исключениями
        lea eax, EXC_1&N
        mov IDT_1&N.OFFS_L, ax
        shr eax, 16
        mov IDT_1&N.OFFS_H, ax
    endm

    lea eax, TIMER_HANDLER                 ;Поместить обработчик
прерывания таймера на 20 шлюз
    mov IDT_TIMER.OFFS_L, ax
    shr eax, 16
    mov IDT_TIMER.OFFS_H, ax

    lea eax, KEYBOARD_HANDLER              ;Поместить обработчик
прерывания клавиатуры на 21 шлюз
    mov IDT_KEYBOARD.OFFS_L, ax

```

```

    shr eax, 16
    mov IDT_KEYBOARD.OFFS_H, ax

    irpc    N, 234567                                ;Заполнить вектора 22-27
заглушками
    lea eax, IDLE_IRQ_MASTER
    mov IDT_2&N.OFFS_L, AX
    shr eax, 16
    mov IDT_2&N.OFFS_H, AX
endm

    irpc    N, 89ABCDEF                                ;Заполнить вектора 28-2F
заглушками
    lea eax, IDLE_IRQ_SLAVE
    mov IDT_2&N.OFFS_L, ax
    shr eax, 16
    mov IDT_2&N.OFFS_H, ax
endm
; ЭТАП 6. Загрузить регистры GDTR и IDTR.
    lgdt fword ptr GDT_GDT                            ;Загрузить регистр GDTR
    lidt fword ptr IDTR                                ;Загрузить регистр IDTR
; ЭТАП 7. Установить бит PE в CR0.
    mov eax, cr0                                        ;Получить управляющий
регистр cr0
    or  al, 00000001b                                  ;Установить бит PE в 1
    mov cr0, eax                                       ;Записать измененный cr0 и
тем самым включить защищенный режим
; ЭТАП 8. Выполнить дальний переход (jmp far) для загрузки CS селектором
защищенного режима.
OVERLOAD_CS:                                          ;Перезагрузить сегмент
кода на его дескриптор
    db  0eah                                           ;Искусственно
сконструированная команда дальнего перехода
    dw  offset OVERLOAD_SEGMENT_REGISTERS              ;Приведет к смене
содержимого IP и CS на селектор дескриптора
    dw  CODE_RM_DESC
; ЭТАП 9. Перезагрузить DS, ES, SS, FS, GS селекторами из GDT.
OVERLOAD_SEGMENT_REGISTERS:                          ;Переинициализировать
остальные сегментные регистры на дескрипторы
    mov ax, DATA_DESC
    mov ds, ax                                        ;В DS находится селектор
дескриптора сегмента данных
    mov es, ax                                        ;Так же как в ES
    mov ax, STACK_DESC
    mov ss, ax                                       ;В SS селектор дескриптора
сегмента стека
    xor ax, ax
    mov fs, ax                                       ;Обнулить регистр fs
    mov gs, ax                                       ;Обнулить регистр gs
    lldt ax                                          ;Обнулить регистр LDTR -
не использовать таблицы локальных дескрипторов
PREPARE_TO_RETURN:                                  ;Подготовится к возврату в
реальный режим
    push cs                                          ;Сохраним сегмент кода в
стек
    push offset BACK_TO_RM                          ;Сохраним смещение точки
возврата
    lea edi, ENTER_PM                               ;Получить точку входа в
защищенный режим

```

mov eax, CODE_PM_DESC	;Получить дескриптор кода
защищенного режима	
push eax	;Занести их в стек
push edi	
; ЭТАП 10. Перепрограммировать PIC на векторы 20h/28h.	
REINITIALIAZE_CONTROLLER_FOR_PM:	;Переинициализировать
контроллер прерываний на вектора 20h, 28h	
mov al, 00010001b	;ICW1 - переинициализация
контроллера прерываний	
out 20h, al	;Переинициализируем
ведущий контроллер	
out 0A0h, al	;Переинициализируем
ведомый контроллер	
mov al, 20h	;ICW2 - номер базового
вектора прерываний	
out 21h, al	;ведущего контроллера
mov al, 28h	;ICW2 - номер базового
вектора прерываний	
out 0A1h, al	;ведомого контроллера
mov al, 04h	;ICW3 - ведущий контроллер
подключен к 3 линии	
out 21h, al	
mov al, 02h	;ICW3 - ведомый контроллер
подключен к 3 линии	
out 0A1h, al	
mov al, 11h	;ICW4 - режим специальной
полной вложенности для ведущего контроллера	
out 21h, al	
mov al, 01h	;ICW4 - режим обычной
полной вложенности для ведомого контроллера	
out 0A1h, al	
mov al, 0	;Размаскировать прерывания
out 21h, al	;Ведущего контроллера
out 0A1h, al	;Ведомого контроллера
; ЭТАП 11. Разрешить прерывания (sti, NMI).	
ENABLE_INTERRUPTS_0:	;Разрешить маскируемые и
немаскируемые прерывания	
in al, 70h	
and al, 01111111b	;Установить 7 бит в 0
для запрета немаскируемых прерываний	
out 70h, al	
nor	
sti	;Разрешить маскируемые
прерывания	
; ЭТАП 12. Перейти в 32-битный код (через push адреса и retf с префиксом 66h).	
GO_TO_CODE_PM:	;Переход к сегменту кода
защищенного режима	
db 66h	;next operation is 32bit
operation	
retf	
BACK_TO_RM:	
режим	;Точка возврата в реальный
cli	;Запрет маскируемых
прерываний	
in al, 70h	;И не маскируемых
прерываний	
or AL, 10000000b	;Установить 7 бит в 1
для запрета немаскируемых прерываний	

out 70h,AL	
nor	
REINITIALISE_CONTROLLER:	;Переинициализация
контроллера прерываний	
mov al,00010001b	;ICW1 - переинициализация
контроллера прерываний	
out 20h,al	;Переинициализируем
ведущий контроллер	
out 0A0h,al	;Переинициализируем
ведомый контроллер	
mov al,8h	;ICW2 - номер базового
вектора прерываний	
out 21h,al	;ведущего контроллера
mov al,70h	;ICW2 - номер базового
вектора прерываний	
out 0A1h,al	;ведомого контроллера
mov al,04h	;ICW3 - ведущий контроллер
подключен к 3 линии	
out 21h,al	
mov al,02h	;ICW3 - ведомый контроллер
подключен к 3 линии	
out 0A1h,al	
mov al,11h	;ICW4 - режим специальной
полной вложенности для ведущего контроллера	
out 21h,al	
mov al,01h	;ICW4 - режим обычной
полной вложенности для ведомого контроллера	
out 0A1h,al	
PREPARE_SEGMENTS:	;Подготовка сегментных
регистров для возврата в реальный режим	
mov GDT_CODE_RM.LIMIT,0FFFFh	;Установка лимита сегмента
кода в 64KB	
mov GDT_DATA.LIMIT,0FFFFh	;Установка лимита сегмента
данных в 64KB	
mov GDT_STACK.LIMIT,0FFFFh	;Установка лимита сегмента
стека в 64KB	
db 0EAh	;Перезагрузить регистр cs
dw offset CONTINUE	
dw CODE_RM_DESC	;На сегмент кода реального
режима	
CONTINUE:	
mov ax,DATA_DESC	;Загрузим сегментные
регистры дескриптором сегмента данных	
mov ds,ax	
mov es,ax	
mov fs,ax	
mov gs,ax	
mov ax,STACK_DESC	
mov ss,ax	;Загрузим регистр стека
дескриптором стека	
ENABLE_REAL_MODE:	;Включим реальный режим
mov eax,cr0	
and al,11111110b	;Обнулим 0 бит регистра
cr0	
mov cr0,eax	
db 0EAh	
dw offset CONTINUE2	
dw CODE_RM	
CONTINUE2:	;Перезагрузим регистр кода
mov ax,STACK_A	

```

    mov ss,ax
    mov ax,DATA
    mov ds,ax
    mov es,ax
    xor ax,ax
    mov fs,ax
    mov gs,ax
    mov IDTR.LIMIT, 3FFH
    mov dword ptr IDTR+2, 0
    lidt fword ptr IDTR
REPAIR_MASK:                                ;Восстановить маски
прерываний
    mov al,INT_MASK_M
    out 21h,al                                ;Ведущего контроллера
    mov al,INT_MASK_S
    out 0A1h,al                                ;Ведомого контроллера
ENABLE_INTERRUPTS:                          ;Разрешить маскируемые и
немаскируемые прерывания
    in al,70h
    and al,01111111b                            ;Установить 7 бит в 0
для разрешения немаскируемых прерываний
    out 70h,al
    nop
    sti                                ;Разрешить маскируемые
прерывания
DISABLE_A20:                                ;Закрыть линию A20
    in al,92h
    and al,11111101b                            ;Обнулить 1 бит -
запретить линию A20
    out 92h, al
EXIT:                                        ;Выход из программы
    mov ax,3h
    int 10H                                    ;Очистить видео-режим
    lea dx,MSG_EXIT
    mov ah,9h
    int 21h                                    ;Вывести сообщение

    call FILL_CR_0_BUFFER_RM                ;Вывести содержимое
регистра CR0
    lea dx, BUFFER_CR_0_RM
    mov ah, 9h
    int 21h

    mov ax,4C00h
    int 21H                                    ;Выход в dos

;Процедура ввода время-нахождения в защищенном режиме ax - результат
INPUT proc near
    mov ah,0ah
    xor di,di
    mov dx,offset ds:[INPUT_TIME]
    int 21h
    mov dl,0ah
    mov ah,02
    int 21h

    mov si,offset INPUT_TIME+2

```

```

        cmp byte ptr [si], "-"                ; пропускаем минус
        jnz ii1
        mov di, 1
        inc si
II1:
        xor ax, ax
        mov bx, 10
II2:
        mov cl, [si]
        cmp cl, 0dh
        jz ii3
        cmp cl, '0'
        jl er
        cmp cl, '9'
        ja er

        sub cl, '0'
        mul bx
        add ax, cx
        inc si
        jmp ii2
ER:
        mov dx, offset MSG_ERROR
        mov ah, 09
        int 21h
        int 20h
II3:
        ret
INPUT endp

FILL_CR_0_BUFFER_RM proc near ; cr0-регистр 0-бит=enables protected mode
        push eax
        push esi
        push dx

        mov eax, cr0 ; запоминаем cr0
        xor dx, dx
        mov cx, 32
        lea esi, BUFFER_CR_0_RM
        fill_cr_0_loop_rm:
        mov dl, al
        shl dl, 7
        shr dl, 7
        shr eax, 1
        add dl, 48
        mov [esi], dl
        inc esi
        xor dl, dl
        loop fill_cr_0_loop_rm

        pop dx
        pop esi
        pop eax
        ret
FILL_CR_0_BUFFER_RM endp

SIZE_CODE_RM      = ($ - CODE_RM_BEGIN)      ; Лимит сегмента кода

CODE_RM ends

```

<pre> ;Сегмент кода защищенного режима CODE_PM segment para use32 CODE_PM_BEGIN = \$ assume cs:CODE_PM,ds:DATA,es:DATA компиляции ENTER_PM: режим call CLRSCR xor edi,edi lea esi,MSG_TITLE_PM protected-mode screen call BUFFER_OUTPUT приветствие в защищенном режиме add edi,160 следующую строку lea esi,MSG_HELLO_PM call BUFFER_OUTPUT mov edi,320 lea esi,MSG_KEYBOARD call BUFFER_OUTPUT скан-кода клавиатуры mov edi,480 lea esi,MSG_TIME call BUFFER_OUTPUT время mov edi,640 lea esi,MSG_COUNT call BUFFER_OUTPUT call FILL_CR_0_BUFFER mov edi, 800 lea esi, MSG_CR0 call BUFFER_OUTPUT mov edi, 960 lea esi, BUFFER_CR_0 call BUFFER_OUTPUT mov DS:[COUNT],0 WAITING_ESC: выхода из защищенного режима jmp WAITING_ESC EXIT_PM: битного сегмента кода db 66H retf сегмент кода EXIT_FROM_INTERRUPT: напрямую из обработчика прерываний popad pop es pop ds pop eax pop eax pop eax </pre>	<pre> ;Указание сегментов для ;Точка входа в защищенный ;Процедура очистки экрана ;В edi смещение на экране ;Header line for the ;Вывести строку- ;Перевести курсор на ;Вывести поле для вывода ;Вывести поле для вывода ;Ожидание нажатия кнопки ;Если был нажат не ESC ;Точка выхода из 32- ;Переход в 16-битный ;Точка выхода для выхода ;Снять со стека старый EIP ;CS ;И EFLAGS </pre>
--	---

```

        sti                                     ;Обязательно, без этого
обработка аппаратных прерываний отключена
        db 66H
        retf                                   ;Переход в 16-битный
сегмент кода

```

```
WORD_TO_DEC proc near
```

```

    pushad
    movzx eax,ax
    xor cx,cx
    mov bx,10

```

```
LOOP1:
```

```

    xor dx,dx
    div bx
    add dl,'0'
    push dx
    inc cx
    test ax,ax
    jnz LOOP1

```

```
LOOP2:
```

```

    pop dx
    mov [di],dl
    inc di
    loop LOOP2
    popad
    ret

```

```
WORD_TO_DEC endp
```

```
FILL_CR_0_BUFFER proc near ; вывести значение cr0 на экран
```

```

    push eax
    push esi
    push dx

```

```

    mov eax, cr0
    xor dx, dx
    mov cx, 32

```

```
; 32 итерации для вывода каждого бита
```

```
регистра
```

```
    lea esi, BUFFER_CR_0
```

```
    fill_cr_0_loop:
```

```
    mov dl, al
```

```
    shl dl, 7 ; получение конкретного бита
```

```
    shr dl, 7
```

```
    shr eax, 1 ; смещаем
```

```
    add dl, 48 ; '0'
```

```
    mov [esi], dl ; кладем результат
```

```
    inc esi ; увеличиваем буфер
```

```
    xor dl, dl
```

```
    loop fill_cr_0_loop

```

```
    pop dx
```

```
    pop esi
```

```
    pop eax
```

```
    ret

```

```
FILL_CR_0_BUFFER endp
```

```

DIGIT_TO_HEX proc near
    add al,'0'
    cmp al,'9'
    jle DTH_END
    add al,7
DTH_END:
    ret
DIGIT_TO_HEX endp

```

```

BYTE_TO_HEX proc near
    push ax
    mov ah,al
    shr al,4
    call DIGIT_TO_HEX
    mov [di],al
    inc di
    mov al,ah
    and al,0Fh
    call DIGIT_TO_HEX
    mov [di],al
    inc di
    pop ax
    ret
BYTE_TO_HEX endp

```

```

M = 0
IRPC N, 0123456789ABCDEF
EXC_0&N label word                                ;Обработчики исключений
00h..0Fh
    cli                                            ;Запрет прерываний
    jmp EXC_HANDLER
endm
M = 010H
IRPC N, 0123456789ABCDEF
EXC_1&N label word                                ;Обработчики исключений
10h..1Fh
    cli                                            ;Запрет прерываний
    jmp EXC_HANDLER
endm

```

```

EXC_HANDLER proc near                                ;Процедура вывода
обработки исключений
    call CLRSCR                                    ;Очистка экрана
    lea esi, MSG_EXC
    mov edi, 40*2
    call BUFFER_OUTPUT                             ;Вывод предупреждения
    pop eax                                        ;Снять со стека старый EIP
    pop eax                                        ;CS
    pop eax                                        ;И EFLAGS
    sti                                            ;Обязательно, без этого
обработка аппаратных прерываний отключена
    db 66h
    retf                                          ;Переход в 16-битный
сегмент кода
EXC_HANDLER      ENDP

```

IDLE_IRQ_MASTER proc near	;Заглушка для аппаратных
прерываний ведущего контроллера	
push eax	
mov al,20h	
out 20h,al	
pop eax	
iretd	
IDLE_IRQ_MASTER endp	
IDLE_IRQ_SLAVE proc near	;Заглушка для аппаратных
прерываний ведомого контроллера	
push eax	
mov al,20h	
out 20h,al	
out 0A0h,al	
pop eax	
iretd	
IDLE_IRQ_SLAVE endp	
TIMER_HANDLER proc near	;Обработчик прерываний
системного таймера	
push ds	
push es	
pushad	;Занести в стек
расширенные регистры общего назначения	
mov ax,DATA_DESC	;Переинициализировать
сегментные регистры	
mov ds,ax	
inc ds:[COUNT]	;Увеличить значение
счетчика	
lea edi,ds:[BUFFER_COUNT]	
mov ax,ds:[COUNT]	
call WORD_TO_DEC	;Преобразовать счётчик в
строку	
mov edi,686	
lea esi,BUFFER_COUNT	
call BUFFER_OUTPUT	;Вывести значение счетчика
SHOW_TIMER:	
mov al,0h	;Выбрать регистр секунд
смос	
out 70h,al	
in al,71h	;Прочитать значение секунд
cmp al,ds:[SECOND]	;Если секунда та же самая
je SKIP_SECOND	;То пропустить вывод
mov ds:[SECOND],al	;Иначе записать значение
новой секунды	
mov al,ds:[TIME]	;Получить значение
оставшегося время	
cmp ds:[TIME],0	;Если время подошло к
концу	
je DISABLE_PM	;То на выход из
защищенного режима	
xor ah,ah	
lea edi,ds:[BUFFER_TIME]	
call WORD_TO_DEC	;Преобразовать его в
строку	
mov edi,522	
lea esi,BUFFER_TIME	

```

        call BUFFER_OUTPUT                ;Вывести значение
оставшегося время
        dec ds:[TIME]                    ;Уменьшить значение
оставшегося времени
        lea esi,BUFFER_TIME
        call BUFFER_CLEAR                ;Очистка буфера
        jmp SKIP_SECOND                  ;На выход из обработки
время
DISABLE_PM:                               ;Выход из защищенного
режима
        mov al,20h
        out 20h,al
        db 0eah                           ;Дальний jmp
        dd OFFSET EXIT_FROM_INTERRUPT    ;На метку
        dw CODE_PM_DESC                   ;В сегменте
SKIP_SECOND:                             ;Секунда та же, не надо
производить никаких действий
        mov al,20h
        out 20h,al                       ;Отправка сигнала
        popad
        pop es
        pop ds
        iretd
TIMER_HANDLER endp

KEYBOARD_HANDLER proc near               ;Обработчик прерывания
клавиатуры
        push ds
        push es
        pushad                            ;Сохранить расширенные
регистры общего назначения
        mov ax,DATA_DESC                  ;Use the program data
segment inside the ISR
        mov ds,ax
        in al,60h                         ;Считать скан код
последней нажатой клавиши
        cmp al,1                          ;Если был нажат 'ESC'
        je KEYBOARD_EXIT                  ;Тогда на выход из
защищенного режима
        mov ds:[KEY_SCAN_CODE],al         ;Записать его в память
        lea edi,ds:[BUFFER_SCAN_CODE]
        mov al,ds:[KEY_SCAN_CODE]
        xor ah,ah
        call BYTE_TO_HEX                  ;Преобразовать скан-код в
строку
        mov edi,374
        lea esi,BUFFER_SCAN_CODE
        call BUFFER_OUTPUT                ;Вывести строку со скан-
кодом
        jmp KEYBOARD_RETURN
KEYBOARD_EXIT:
        mov al,20h
        out 20h,al
        db 0eah
        dd OFFSET EXIT_FROM_INTERRUPT
        dw CODE_PM_DESC
KEYBOARD_RETURN:

```

mov al,20h	
out 20h,al	;Отправка сигнала
контроллеру прерываний	
popad	;Восстановить значения
регистров	
pop es	
pop ds	
iretd	;Выход из прерывания
KEYBOARD_HANDLER endp	
CLRSCR proc near	;Процедура очистки консоли
push es	
pushad	;запустить все регистры
общего назначения	
mov ax,TEXT_DESC	;Поместить в ax дескриптор
текста	
mov es,ax	
xor edi,edi	
mov ecx,80*25	;Количество символов в
окне	
mov ax,1F20h	;Bright white on blue
background	
rep stosw	
popad	
pop es	
ret	
CLRSCR endp	
BUFFER_CLEAR proc near	;Процедура очистки буфера,
в esi кладем буфер	
mov al,' '	
mov [esi+0],al	
mov [esi+1],al	
mov [esi+2],al	
mov [esi+3],al	
mov [esi+4],al	
mov [esi+5],al	
mov [esi+6],al	
mov [esi+7],al	
ret	
BUFFER_CLEAR endp	
BUFFER_OUTPUT proc near	;Процедура вывода
текстового буфера, оканчивающегося 0	
push es	
PUSHAD	
mov ax,TEXT_DESC	;Поместить в es селектор
текста	
mov es,ax	
OUTPUT_LOOP:	;Цикл по выводу буфера
lodsb	
or al,al	
jz OUTPUT_EXIT	;Если дошло до 0, то конец
выхода	
stosb	
inc edi	
jmp OUTPUT_LOOP	

```

OUTPUT_EXIT:                                ;Выход из процедуры вывода
    popad
    pop es
    ret
BUFFER_OUTPUT ENDP

SIZE_CODE_PM      =      ($ - CODE_PM_BEGIN)
CODE_PM  ENDS

;Сегмент данных реального/защищенного режима
; DATA SEGMENT 8
DATA      segment para use16                ;Сегмент данных
реального/защищенного режима
DATA_BEGIN      = $
    ;Структуры данных

; _____
;
; S_DESC  struc                                ;Структура сегментного
; дескриптора
;     LIMIT      dw 0                        ;0,1 - Лимит сегмента
;     BASE_L      dw 0                        ;2,3 - Адрес базы,
; младшая часть
;     BASE_M      db 0                        ;4 - Адрес базы,
; средняя часть
;     ACCESS      db 0                        ;5 - Байт доступа
;     ATTRIBS     db 0                        ;6 - Лимит сегмента и
; атрибуты(7: G(дробность) G=0 - в байтах, G=1 - в страницах; 6:
; D/B(разрядность) - =0: 16/8, =1: 32/8; 5: always 0; 4: avail; 3-0: лимит)
;     BASE_H      db 0                        ;7 - Адрес базы,
; старшая часть
;     S_DESC  ends
;     I_DESC  struc                                ;Структура дескриптора
; таблицы прерываний
;     OFFS_L      dw 0                        ;Адрес обработчика
; (0:15)
;     SEL         dw 0                        ;Селектор кода,
; содержащего код обработчика
;     PARAM_CNT   db 0                        ;Параметры
;     ACCESS      db 0                        ;Уровень доступа
;     OFFS_H      dw 0                        ;Адрес обработчика
; (31:16)
;     I_DESC  ends
;     R_IDTR  struc                                ; IDTR structure:
;     LIMIT      dw 0                        ; 16-Bit Table Limit
; 0..15
;     IDT_L      dw 0                        ; 32-Bit Linear Base
; Address (low) 16..31
;     IDT_H      dw 0                        ; 32-Bit Linear Base
; Address (high) 32..47
;     R_IDTR  ends

; _____
;
; _____
;
; GDT - глобальная таблица дескрипторов

```

```

;
GDT_BEGIN = $
GDT_label word ;Метка начала GDT
GDT_0 S_DESC <0,0,0,0,0,0>
GDT_GDT S_DESC <GDT_SIZE-1,,,10010010b,0,> ; 7: p = 1 -
присутствие в памяти; 5-6: DPL = 0 - уровень защиты(макс) ; 4: s = 1 - тип
сегмента(сегмент кода или данных) ; 3: сегмент данных; 2: e = 0 -
направление расширения(вверх по адресам) ; 1: w = 1 - writeable(разрешается
запись(и чтение)) ; 0: a = 0 - востребованность сегмента(на магнитный диск)
GDT_CODE_RM S_DESC <SIZE_CODE_RM-1,,,10011010b,0,> ; 7: p = 1 -
присутствие в памяти; 5-6: DPL = 0 - уровень защиты(макс) ; 4: s = 1 - тип
сегмента(сегмент кода или данных) ; 3: сегмент программ; 2: c = 0 -
confirming(разрешает обращение только когда CPL=DPL) ; 1: r = 1 -
readable(разрешается считывание) (редактировать нельзя, чтобы
редактировать, нужно создать сегмент данных с тем же адресом с w = 1, и
работать через него); 0: a = 0 - востребованность сегмента(на магнитный
диск)
GDT_DATA S_DESC <SIZE_DATA-1,,,11110010b,0,> ; 7: p = 1 -
присутствие в памяти; 5-6: DPL = 11 - уровень защиты(мин) ; 4: s = 1 - тип
сегмента(сегмент кода или данных) ; 3: сегмент данных; 2: e = 0 -
направление расширения(вверх по адресам) ; 1: w = 1 - writeable(разрешается
запись(и чтение)) ; 0: a = 0 - востребованность сегмента(на магнитный диск)
GDT_STACK S_DESC <1000h-1,,,10010010b,0,> ; 7: p = 1 - присутствие
в памяти; 5-6: DPL = 0 - уровень защиты(макс) ; 4: s = 1 - тип
сегмента(сегмент кода или данных) ; 3: сегмент данных; 2: e = 0 -
направление расширения(вверх по адресам) ; 1: w = 1 - writeable(разрешается
запись(и чтение)) ; 0: a = 0 - востребованность сегмента(на магнитный диск)
GDT_TEXT S_DESC <2000h-1,8000h,0Bh,11110010b,0,0>
GDT_CODE_PM S_DESC <SIZE_CODE_PM-1,,,10011010b,01000000b,> ; 32/8 в
байтах, ; 7: p = 1 - присутствие в памяти; 5-6: DPL = 0 - уровень
защиты(макс) ; 4: s = 1 - тип сегмента(сегмент кода или данных) ; 3:
сегмент данных; 2: e = 0 - направление расширения(вверх по адресам) ; 1: w
= 1 - writeable(разрешается запись(и чтение)) ; 0: a = 0 - востребованность
сегмента(на магнитный диск)
GDT_IDT S_DESC <SIZE_IDT-1,,,10010010b,0,> ; 7: p = 1 -
присутствие в памяти; 5-6: DPL = 0 - уровень защиты(макс) ; 4: s = 1 - тип
сегмента(сегмент кода или данных) ; 3: сегмент данных; 2: e = 0 -
направление расширения(вверх по адресам) ; 1: w = 1 - writeable(разрешается
запись(и чтение)) ; 0: a = 0 - востребованность сегмента(на магнитный диск)
GDT_SIZE = ($ - GDT_BEGIN) ;Размер GDT

```

```

;
;Селлекторы сегментов
CODE_RM_DESC = (GDT_CODE_RM - GDT_0)
DATA_DESC = (GDT_DATA - GDT_0)
STACK_DESC = (GDT_STACK - GDT_0)
TEXT_DESC = (GDT_TEXT - GDT_0)
CODE_PM_DESC = (GDT_CODE_PM - GDT_0)
IDT_DESC = (GDT_IDT - GDT_0)

;IDT - таблица дескрипторов прерываний
;
IDTR R_IDTR <SIZE_IDT,0,0> ;Формат регистра IDTR
IDT_label word ;Метка начала IDT
IDT_BEGIN = $
IRPC N, 0123456789ABCDEF

```

```

        IDT_0&N I_DESC <0, CODE_PM_DESC,0,10001111b,0>          ;
00...0F
    ENDM
    IRPC      N, 0123456789ABCDEF
        IDT_1&N I_DESC <0, CODE_PM_DESC, 0, 10001111b, 0>      ;
10...1F
    ENDM
    IDT_TIMER      I_DESC <0, CODE_PM_DESC,0,10001110b,0>      ;IRQ 0
- прерывание системного таймера
    IDT_KEYBOARD I_DESC <0, CODE_PM_DESC,0,10001110b,0>      ;IRQ 1
- прерывание клавиатуры
    IRPC      N, 23456789ABCDEF
        IDT_2&N      I_DESC <0, CODE_PM_DESC, 0, 10001110b, 0>  ;
22...2F
    ENDM
    SIZE IDT      =      ($ - IDT_BEGIN)
    ; Размер IDT
    ;

MSG_HELLO      db "Press any key to enter protected
mode",13,10,"$"
MSG_TITLE_PM   db "=== Protected Mode Lab: Timer IRQ + Keyboard
IRQ Demo ===",0
MSG_HELLO_PM   db "ESC exits immediately. Timer countdown returns
to real mode.",0
MSG_EXIT       db "Returned to real mode successfully.",13,10,"$"
MSG_KEYBOARD   db "Last keyboard scan code:      ",0
MSG_TIME       db "Remaining time in PM: XXXXXXXX seconds",0
MSG_COUNT      db "Timer interrupt calls: ",0
MSG_CR0        db "CR0 register bits:",0
MSG_EXC        db "Processor exception: XX",0
MSG_ENTER      db "Enter protected-mode countdown in seconds: $"
MSG_ERROR      db "Input error$"
HEX_TAB        db "0123456789ABCDEF"      ;Таблица номеров
исключений
ESP32          dd 1 dup(?)      ;Указатель на вершину
стека
INT_MASK_M     db 1 dup(?)      ;Значение регистра масок
ведущего контроллера
INT_MASK_S     db 1 dup(?)      ;Значение регистра масок
ведомого контроллера
KEY_SCAN_CODE  db 1 dup(?)      ;Ска-код последней нажатой
клавиши
SECOND         db 1 dup(?)      ;Текущее значение секунд
TIME           db 1 dup(10)     ;Время нахождения в
защищенном режиме
COUNT         dw 1 dup(0)      ;Количество вызовов
прерывания (диапазон от 0 до 65535)
BUFFER_COUNT    db 8 dup(' ')   ;Буфер для вывода
количества вызовов прерываний
              db 1 dup(0)
BUFFER_SCAN_CODE db 8 dup(' ')   ;Буфер для вывода скан-
кода клавиатуры
              db 1 dup(0)
BUFFER_TIME     db 8 dup(' ')   ;Буфер для вывода
оставшегося время в защищенном режиме
              db 1 dup(0)
INPUT_TIME      db 6,7 dup(?)   ;Буфер для ввода время
BUFFER_CR_0     db 32 dup('?')

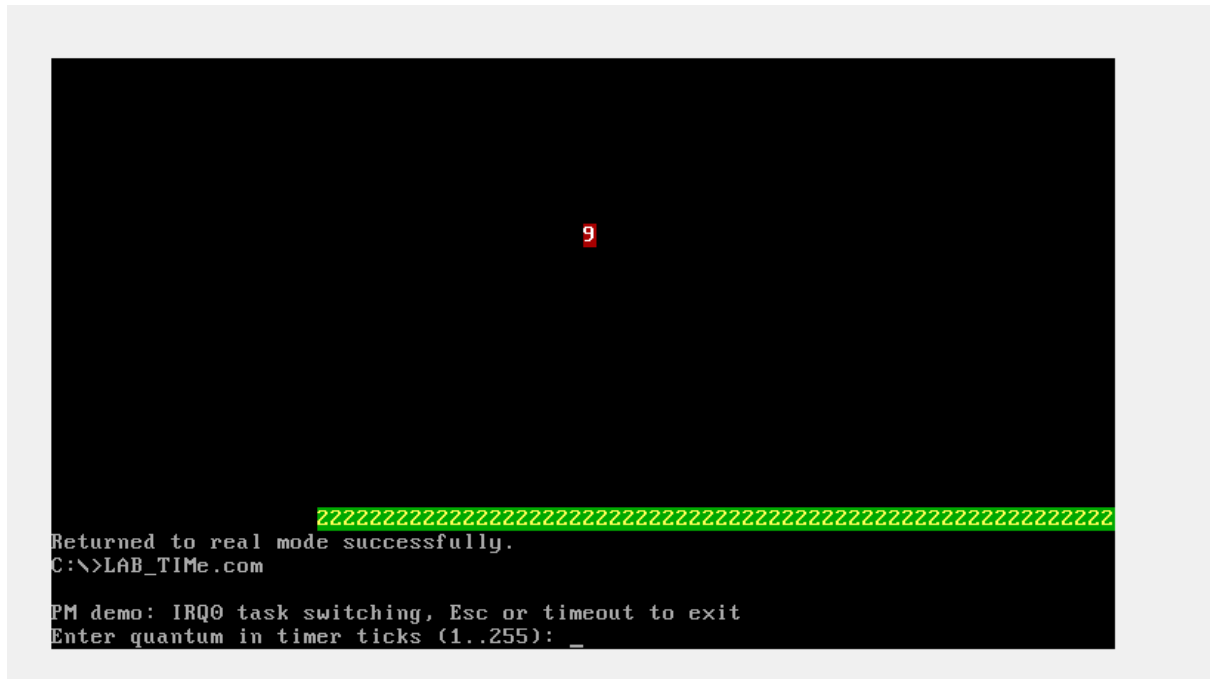
```

```

                                db 1 dup(0)
    BUFFER_CR_0_RM              db 32 dup('?'), 13, 10, "$"    ; буфер
для вывода значения регистра cr0
SIZE_DATA    = ($ - DATA_BEGIN)                ;size of data segment
DATA        ends
;Сегмент стека реального/защищенного режима
STACK_A segment para stack
    db 1000h dup(?)
STACK_A ends
end START

```

Результаты работы



1.1 Ввод



1.2 Работа в защищенном режиме

Вывод:

В ходе выполнения лабораторной работы была разработана программа для MS-DOS, реализующая переход процессора из реального режима в защищённый по алгоритму, используемому в процессорах семейства 80386 и выше, а затем выполняющая обратный переход в реальный режим с корректным завершением работы.

В программе был реализован перехват аппаратных прерываний от клавиатуры и системного таймера. Обработчик прерывания таймера выполнял подсчёт вызовов, отсчитывал секунды и выводил их на экран. Обработчик прерывания клавиатуры считывал скан-коды нажатых клавиш и также выводил их на экран. В качестве события завершения работы использовалось либо истечение заданного пользователем времени, либо нажатие выбранной клавиши.

В процессе выполнения лабораторной работы были изучены:

- особенности работы процессора в реальном и защищённом режимах;
- порядок включения protected mode через установку бита PE регистра CR0;
- использование таблиц GDT и IDT;

- принципы обработки аппаратных прерываний;
- особенности взаимодействия с контроллером прерываний PIC;
- организация корректного возврата из защищённого режима в реальный без нарушения работы системы.

В результате была подтверждена возможность организации собственной обработки аппаратных прерываний в защищённом режиме с последующим безопасным восстановлением исходного состояния компьютера. Поставленная цель лабораторной работы была достигнута.